17. Networking Basics: IP, DNS, Ports

17. Networking Basics: IP, DNS, Ports

Phase: Phase 6 — Networking
Estimated time: 1–2 days
Slug: `17-networking-basics`

Learning objectives

- Explain IP, CIDR, ports at a practical level
- Query DNS with dig/nslookup/getent
- Inspect addresses with ip
- Test connectivity with ping/curl

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Nearly every outage ticket contains the words “cannot connect”. To debug one you need a mental model of what sits between two machines: an IP address to find the host, DNS to turn names into addresses, a port to pick the service, and a route to get packets there. If you can name the layer that is broken, the fix is usually one command away.

This lesson gives you the minimum vocabulary and the commands to inspect each layer on a Linux host: `ip`, `resolvectl` or `dig`, `getent`, `ping`, and the well-known port numbers, such as 443 for HTTPS.

Core concepts

Concept	Example	Meaning
IPv4 address	`192.168.1.20/24`	32-bit host address plus prefix length
IPv6 address	`2001:db8::1`	128-bit address
Gateway	`192.168.1.1`	Next hop for non-local traffic
DNS	`example.com -> 93.184.216.34`	Name to address mapping
Port	`443`	16-bit number selecting a service on a host
Protocol	TCP / UDP	Reliable stream versus datagrams
Socket	`10.0.0.5:443`	IP plus port plus protocol
`/etc/resolv.conf`	`nameserver 127.0.0.53`	Where the resolver asks
`/etc/hosts`	`127.0.0.1 localhost`	Local static name overrides

Well-known ports:

Port	Service
22	SSH
53	DNS
80	HTTP
443	HTTPS
25	SMTP
5432	PostgreSQL

What happens when you open https://example.com:

```
browser/curl
   |  1. name lookup: example.com ?
   v
resolver --(/etc/hosts, then DNS server)--> 93.184.216.34
   |
   |  2. route lookup: not on my subnet -> send to gateway
   v
gateway --> internet --> server 93.184.216.34
   |
```

```
    |  3. TCP connect to port 443 (default https port)
    v
  TLS handshake, then HTTP request
```

Worked examples

Show addresses and routes:

```
ip -br addr
ip route
```

Sample output:

```
lo        UNKNOWN  127.0.0.1/8 ::1/128
eth0      UP       192.168.1.20/24 fe80::a00:27ff:fe4e:66a1/64
default via 192.168.1.1 dev eth0 proto dhcp metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.20
```

Resolve a name the way applications do (honours `/etc/hosts` and NSS):

```
getent hosts example.com
dig +short example.com
```

Sample output:

```
2606:2800:21f:cb07:6820:80da:af6b:8b2c example.com
93.184.216.34
```

Look up a service port from the system database:

```
getent services https
grep -w 443 /etc/services | head -2
```

Sample output:

```
https                443/tcp
https           443/tcp
https           443/udp         # http protocol over TLS/SSL
```

Reachability and path:

```
ping -c 2 192.168.1.1
tracepath -n example.com | head -3
```

Sample output:

```
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.612 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.540 ms
 1?: [LOCALHOST]                      pmtu 1500
 1:  192.168.1.1                      0.713ms
```

A shell helper for the exercise idea:

```
default_https_port() { echo 443; }
default_https_port
```

Sample output:

```
443
```

Mini-lab

1. Run `ip -br addr` and write down your IPv4 address and prefix length.
2. Run `ip route` and identify the default gateway.
3. Ping the gateway, then ping `1.1.1.1`, then ping `example.com`.
4. Edit nothing, but read `/etc/resolv.conf` and `/etc/hosts`.
5. Compare `getent hosts example.com` with `dig +short example.com`.
6. Add a temporary line to `/etc/hosts` for `lab.test` pointing at `127.0.0.1` (needs sudo), run `getent hosts lab.test`, then remove it.
7. Use `getent services ssh http https` and fill a three-row table of service and port.
8. Use `tracepath -n example.com` and count the hops.

Decision guide / failure modes

Symptom	Layer	Check
`ping 1.1.1.1` fails	Link / route	`ip addr`, `ip route`, cable or Wi-Fi
Ping IP works, name fails	DNS	`resolvectl status`, `dig`
Name resolves, connect refused	Service / port	Is the daemon listening?
Name resolves, connect times out	Firewall or routing	Security group, `iptables`, route
Works by IP, wrong host by name	`/etc/hosts` override	`getent hosts name`
Ping blocked but service up	ICMP filtered	Test the real TCP port

Work bottom-up: address, route, DNS, port, application. Refused means a host answered and said no; timeout means nothing answered. Remember that `ping` proves only ICMP; use the real port for the real answer, and that the default port for https is 443 and for http is 80.

Common mistakes

- Assuming ping failure means everything is down (ICMP may be blocked)
- Hardcoding IPs without DNS strategy
- Opening all ports “temporarily”

Exercises

Work in `exercises/17-networking-basics/`. Read that folder’s `README.md` fully before coding.

```
cd exercises/17-networking-basics
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement default_https_port in exercise.sh so it prints the standard TCP port used by HTTPS, with nothing else on the line. It takes no arguments. Use echo and a plain number; the test compares the output exactly, so avoid extra spaces or text such as “port”. When done, confirm the value against the system database with getent services https and read how the port number, protocol and service name relate. Optionally, write a second helper of your own for http (80) and ssh (22) to practice the table from the lesson.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/17-networking-basics/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is a port?
2. How resolve DNS from CLI?
3. ip addr shows what?

Further reading

- https://man7.org/linux/man-pages/man8/ip.8.html